AI Agentic Engineering Guide

Design them the way you
agents share state? How
between, an explicit
model only fills in each step.
ommarize ten documents at
steps: # LangGraph
3. Autonomous patterns:
he results, and retries failures, like
t back and forth with an one
or it only when a fixed rotor cannot
has always used for parallel work, a
exactly one agent (Task 1 to a, Task 2
s where multi-agent systems break.
Use autonomy. The answer shows up
You end up with a mostly-explicit
the evaluation before you scale the
with code (exact match, a schema, a
t will quietly drift. 3. Run the simplest
simple task, one model call often wins.
nstructions. The prompt and the
nside a single agent is a loop. Understand it, and
he work is done or a limit is hit. - Verify: before
plan its next step between tool calls. A healthy loop
together. One message runs it all, and a human only at
Everything after is automatic. - Report block: an agent
back to the human. During testing, plant a fault on
building the same deliverable. A copy each, no shared
procedure Do this, in order: 1. Define good: write the test
5. Find the one step that cannot be
worktree). add claims, and merge
cap every loop, handle every failure.
(accuracy, structure, writing). If you cannot
source list (sources). Check the output for

Multi-agent design software, and add
they share what they know. Each
skeleton with intelligence at the joints.
Three shapes cover almost everything. ###
once, then combine. ### Conditional fixed the
(graph.py) graph.add_edge("draft", "check") #
emergent control, routed by the model like
a project manager. Risk: the lead is a single point of
finishing. ### Group chat All agents share
express the task. ## 4. State: give each
copy each plus a system around it: - A
te 15). Its two agents isolate it, claim it,
ency by trying. And the intelligent joints, which is
pipeline and one or two test jobs first. Decide what
system. 1. Write the judge when it is a
passing test). Use a complex one. In one
version alongside the cannot justify with a
Do not add agents you guidelines each agent followers. Sharper
bound it. - Gather context: read the task,
finishing. the agent checks its own work.
is: gather context, take action, re-run the
the end. - Intake: one message (from a
gathers sources, does the work, and runs its
purpose (a quote changed to claim more than it says:
state. - Human at the end: the person picks the best
obs, including the awkward cases. 2. Start with
pre-scripted. Make only that step an
task. 7. Add a judge for judgment-cal
11. Prompts you can reuse Worker
finish within (ii) steps, stop and
accuracy (every

Systems A practical, but deeply quick to get restrained AI
intelligence only where it earns its place. ## 1.
answer sits on a spectrum. At one end is
know where each part of your system sits
Sequential One step after another. Use it when
request and pick the route. Use it when the path
sequential: draft, then check
autonomy when you do NOT know the steps,
ailure. If its plan is wrong, everything
one conversation. Easy to wire up. Risk: as the
agent its own copy
Merges. Each
merge is. ## 5. How to choose One question
rate that matters most: even then, start with
cheaper, more reliable, and easier to debug
good looks like, and include the awkward
udgement call: a second model reviews the
published text, a single model call beat a
measured gain. ## 7. Optimisation: where
nstructions fix more than extra agents do.
atch what you need. -
- Turn-limit guard: cap
oop, verify the work,
phone, Slack, or a ticket) creates a card on a task board
own checks for accuracy, structure, and
and confirm the judge catches it. - Fan-out: split
of the several outputs. That is the only human
one model call. Measure quality, tokens, and
autonomous agent, and pick its pattern
outputs, and grade the judge. 8. Optimise
agent: "You are (role). Goal: (task). Tools
report exactly what is blocking you. Do not
quote and number must appear in the source,

recently, written for a
The two questions
EXPLICIT CONTROL: you
before you build it. ## 2.
the steps are known and
depends on the input: a
graph.add_conditional_edge
when the path only shows
downstream is wrong. ###
conversation grows, cost
The check: two agents both
agent its own workspace, a git
workspace merges back to the
decides the shape: do you know the steps? - Yes:
a workflow. Lay down the steps you do know, run it,
than autonomy everywhere. ## 6. Evaluation: is it
cases, the inputs that are easy to get wrong. 2.
output. And check the judge too, grade the grader
team of agents on every axis: quality 9.7 against 9.3,
most of the value is When the system works but is
2. Tools. What each agent can actually do. The right
Take action: the model requests tool calls; you
the iterations, and raise a clear "max turns
with a turn limit so it cannot run away. ## 9.
with three columns, NEW, PICKED UP,
writing. Then a judge, a second agent
the work into several isolated
step. - Everything is agged: a live event stream
calls. 3. If one call is enough, skip it. It often is,
(plan-based, handoff, or group chat)
instructions first, then tools. 9. Keep a
available: (tools). Gather the context you
guess." Judge agent: "You are a
unchanged), structure (required sections), and

machine to test and
Every multi-agent
decide everything
Workflow patterns:
ordered: draft, then
(step, step, then
", route) #
up by trying. Each
Handoff Each agent
climbs, details are
write the same file,
worktree (wt-a,
main line. Conflicts
judge when it is a
complex one. In one
cannot justify with a
followers. Sharper
context: read the task,
checks its own work.
action, re-run the
message (from a
the work, and runs its
purpose (a quote changed to claim more than it says:
state. - Human at the end: the person picks the best
obs, including the awkward cases. 2. Start with
pre-scripted. Make only that step an
task. 7. Add a judge for judgment-cal
11. Prompts you can reuse Worker
finish within (ii) steps, stop and
accuracy (every

jet on. The claim throughout these systems are old metric. They are
design is the answer to two questions: 1. Who controls
up front, in code. At the other is EMERGENT CONTROL:
explicit control, routed in code Use a workflow when
check, then publish. ### Parallel Fan out, then bring
sends a ticket to the right handler. Frameworks such
conditional: triage picks the route ''' If you can draw
pattern has a failure mode: know it before you choose
decides who gets the work next: desk, then billing,
ost, and decisions conflict. Many voices, one thread,
guide.md. One
wt-b). They never write
are settled at merge
explicit control. Use a
and watch where it
working? You cannot
Grade two ways. Use
against a few
tokens 355 against
not good enough,
tool, well described,
executs them and feed
exceeded" rather than
Reference architecture:
DONE. - Listener:
seeing the work for the
sandboxes, each
and an explicit per-step
4. If not, draw the
with its failure-mode
human at the end for the
need, then do the work.
reviewer seeing this
writing (rules)). List each

gentlest software (on) with
the flow? Who decides
the model decides at
you already know the
the results back. Use it
as LangGraph express
the graph, write it in
it. ### Plan-based A
then tech. Flexible.
no owner. Autonomy
overwrites the other,
over each other,
time, and the work
workflow (sequential,
breaks. Replace only
improve what you
code when there is a
known-good and
15,243, model calls 3
optimise in this order.
beats a clever plan
the results back. - Run
loop forever. - Plan
a working system One
watches the board,
first time, reads the
running a different
status, so you can see
workflow (sequential,
guard. 6. If agents write
pick or the approval.
before you finish, verify
work for the first time.
problem and a concrete

A component.
what happens next. 2. How do
run time. Most real systems live in
steps. The route is fixed in code; the
(or parts that do not depend on each other:
exactly this. A workflow is just a graph of
code. The model is a node, not the orchestrator.
lead agent plans the work, hands out tasks, checks
Risk: the work can go in circles, agents passing
buys flexibility, and costs control. Reach
and work is lost. The fix and verify is the one software
Claims. Each unit of work is claimed by
comes back together. Shared mutable state
parallel, conditional). - No emergent control.
The broken step with an autonomous agent.
cannot measure. Build
right answer: check it
known-bad examples, or
against 3 or more. For a
not in more agents. 1.
Most of the value is here,
5. The agent loop
over the wrong tools. ##
gather-and-act until
the loop: repeat
tasks, let the model
between calls: for hard
puts the patterns
concrete system that
and starts the flow.
picks up a new card,
report. After a few fix cycles It is handed
model (for example Opus, Sonnet, and a GPT model)
what each agent did and why. ## 10. Build
parallel, conditional) and write the route in code.
shared artifacts. give each its own copy (a
10. Treat it as software: log every step,
your output against these checks:
The task wrac (task): The
fix. Do not rewrite